MoveInSync Backend Case Study 2026

SDE – Java Backend Hiring Assessment · Advanced Track (IIIT & NIT) · Case Study 4

Case Study 4: Live Trip Updates Fan-Out – Ordered, Resumable, Back-Pressured

Security desks, transport admins and employees watch trips live: cab location, trip status changes, delays and SOS alerts. At peak there are 20,000 cabs sending updates and 200,000 connected screens. An SOS must reach the security desk within 2 seconds, every time, even when the system is busy. Your job is to build the publish-subscribe layer that delivers these updates over WebSocket.

I. Topics & Access

- Topics — per cab, per trip and per office.
- Authorisation per Subscription — an employee can subscribe only to their own trip, an office admin only to their office, and nobody to another client's data. Check this on subscribe, not just on login.

II. Delivery Guarantees

- Per-Key Ordering — events for one trip are always delivered in order.
- At-Least-Once with Dedup — every event has a per-topic sequence number, and clients drop duplicates.
- Resume — a client that reconnects with its last sequence number receives only what it missed from a bounded replay buffer. If it was away too long, it gets a fresh snapshot followed by live events.

III. Slow Consumers & Priority

- Back-Pressure — a slow client on a bad network must never slow down other clients. Each connection has a bounded queue.
- Conflation — for location updates keep only the latest one per cab when a client falls behind. Status changes and SOS events are never dropped.
- Priority Lane — SOS events skip ahead of location traffic everywhere in the pipeline.

IV. Scale-Out & Failure

- Many Gateway Nodes — connections are spread across several nodes. Explain how an event reaches the node that holds a given subscriber.
- Node Crash — when a node dies, its clients reconnect to another node and resume without losing a status or SOS event.

Example Use Case

• An employee's phone goes into a lift for 40 seconds. On reconnect it receives the three status changes it missed, in order, and only the latest location.

• One security desk browser freezes. Its queue fills and its location updates are conflated, while SOS latency for every other desk stays under 2 seconds.

Key Points to Keep in Mind

- Ordering, dedup and resume all depend on the sequence number. Define who assigns it and where.
- State the memory cost per connection and the replay-buffer size, and what happens when either runs out.
- Tenant isolation on subscribe is a security requirement. Include a test that tries to subscribe to another client's trip.
- **Prove it:** simulate at least 10,000 connections and 2,000 publishing cabs, report SOS p99 latency, and kill one gateway node during the test.

Plus Points in Implementation (Overall Evaluation Criteria)

1. **Authentication:** Implement robust authentication and role-based access (employee vs admin vs operator), with tenant isolation so one client can never read another client's data.
2. **Cost Estimation – Time and Space:** Analyse the time and space complexity of your core logic, and back it with a small benchmark. State where the system stops scaling and why.

3. **Handling System Failure Cases:** Implement fault-tolerant mechanisms for crashes, timeouts, duplicate and out-of-order messages, and partial failures. Show at least one failure-injection test.
4. **Object-Oriented Programming (OOPS):** Choose a robust OOPS language (Java preferred). Apply encapsulation, inheritance and polymorphism for maintainable, extensible code with clean controller / service / repository layering.
5. **Trade-offs in the System:** Clearly define and document the trade-offs you made, especially consistency vs availability vs latency. Explain what you deliberately left out.
6. **System Monitoring:** Add metrics and structured logging. Expose the counters an on-call engineer would need at 3 AM (lag, error rate, rejections, retries, queue depth).
7. **Caching:** Integrate caching for hot data with a sensible eviction policy. Say exactly what can become stale, for how long, and how staleness is bounded.
8. **Error and Exception Handling:** Build a robust error-handling framework with meaningful, non-leaking error messages. No personal data (phone, email, home location) in logs or error responses.

What “Advanced Track” Means

- **Proven guarantees** – every hard rule in your case study has an automated test, including at least one concurrency test and one failure test.
- **Measured performance** – a small load test or benchmark with throughput and p50 / p99 latency, reported in the README with the machine you ran it on.
- **Design note** – one or two pages with an architecture diagram, one sequence diagram for the hardest flow, and a failure-mode table (what fails, what the user sees, how the system recovers).
- **One-command run** – the project starts with a single command (for example docker compose up), including any database or message broker it needs.

Instructions

1. **Read and Understand the Problem Statement:** Carefully read the case study. Understand the requirements, inputs, expected outputs, the hard rules and the scale targets.
2. **Choose a Programming Language:** Java with Spring Boot is preferred. C#, Go, Kotlin or Python are acceptable if you can defend the choice.
3. **Design Your Solution:** Plan first. Decide your data model, the main components, the APIs and the failure modes before writing code. Put this in the design note.
4. **Write the Code:** Write clean, layered code with meaningful names. Keep the core algorithm in plain classes that can be tested without a web server or database.
5. **Test Your Code:** Test normal inputs AND edge cases. Add unit tests for the core logic, plus the concurrency, failure and benchmark tests described in your case study.
6. **Document Your Code:** Add a README with how to run it, your assumptions, the benchmark results, and a short note on the trade-offs you made.
7. **Submit Your Solution:** Push to a public GitHub repository and share the link. Keep a meaningful commit history, not one giant commit.
8. **Demonstration:** Add a short demo video showing the main flows, one failure scenario being handled live, and a few sample API calls (Postman / curl).